

ARCHITECTURE DESIGN DOCUMENT

Quest

AI Support Agent for Dynamics 365 / ServiceNow

Automated triage, root-cause diagnosis, grounded recommendation, and guarded auto-remediation of support cases — with a dual reasoning engine, an MCP tool layer, and a safety net that suggests rather than acts when in doubt.

FastAPI · Python

PostgreSQL + pgvector

Redis

Azure OpenAI · gpt-4o

Model Context Protocol

Dataverse Web API

Repository: [ramya-yalamarthi/quest](#)

Technical Architecture · v1.0

§ Contents

1. Executive summary
2. System context
3. High-level architecture
4. Component & layer architecture
5. Ticket lifecycle — state machine
6. End-to-end data flow
7. Dual reasoning engine
8. Agents, roster & SLA
9. MCP (Model Context Protocol) layer
10. Data model
11. External integrations & graceful degradation
12. Safety net — guarded auto-remediation
13. Deployment topology
14. Key design decisions & roadmap

1. Executive summary

When a support Case is created, transferred, or reactivated in Dynamics 365 or ServiceNow, Quest analyses it against the team's real case history and posts one grounded recommendation directly onto the Case — hands-off, 24/7.

① Route

Assign the right support team and pick an available, skill-matched, on-shift engineer.

② Diagnose

Root cause grounded in the closest *historically resolved* cases via semantic similarity.

③ Recommend

Hot Fix (restore now) + Ultimate Fix (permanent) with allowlisted reference links.

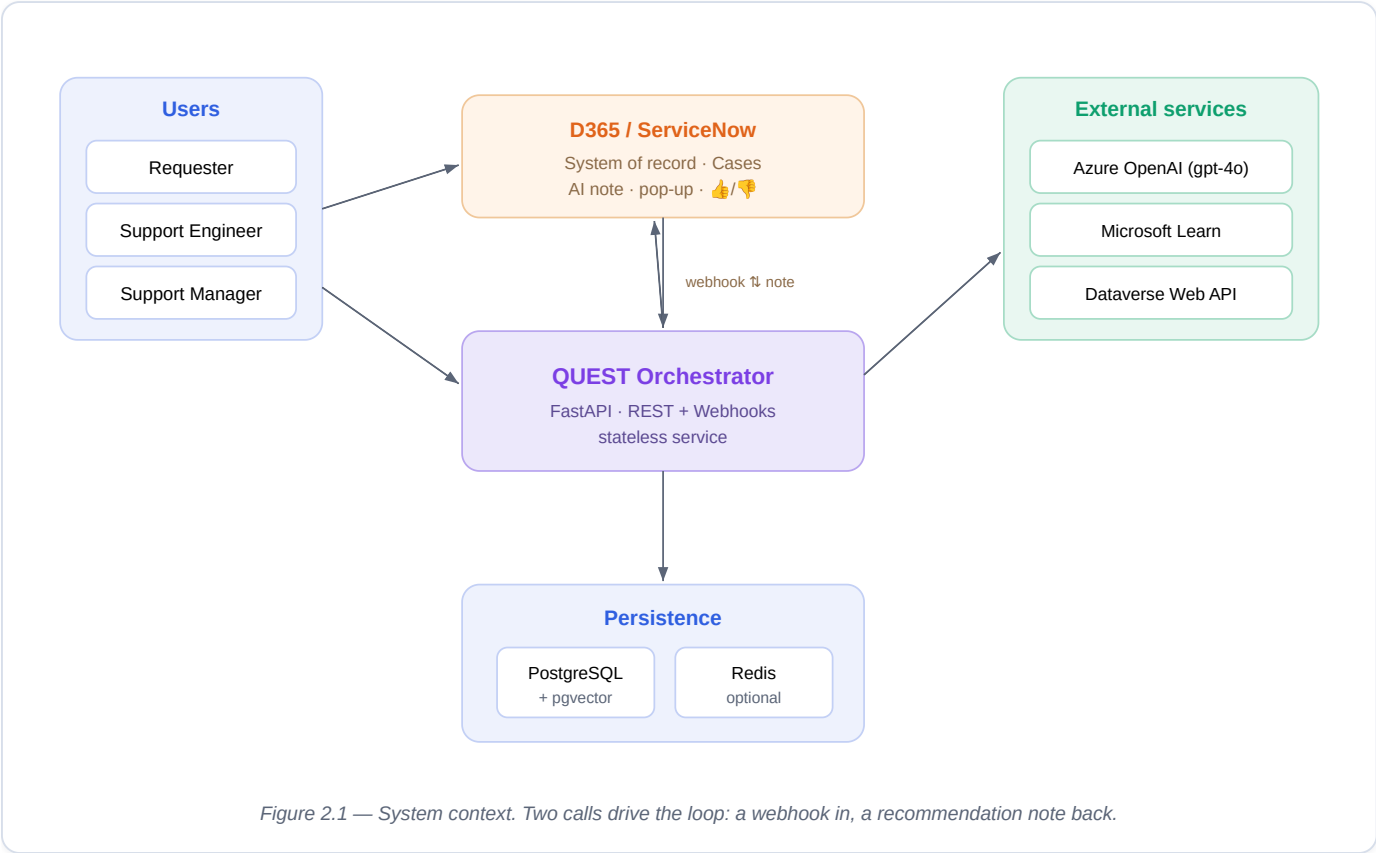
④ Remediate (guarded)

Low-risk, high-confidence cases: simulate → verify → revert-if-failed, with a kill switch.

Two principles run through everything: reasoning is always *grounded* in real past cases (never hallucinated), and *every external dependency is optional* — a failure in any one stage degrades that stage gracefully instead of failing the case.

2. System context

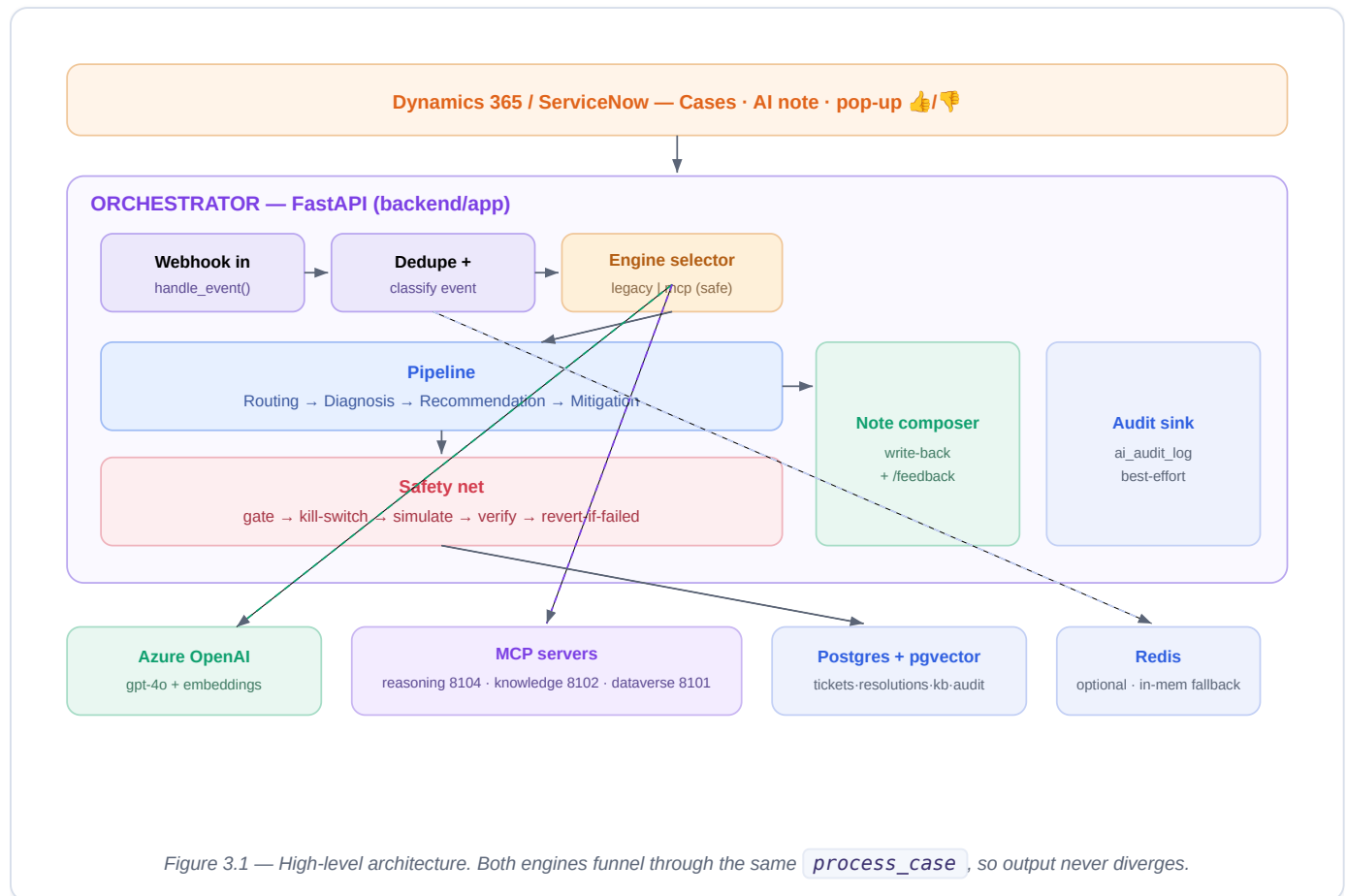
Quest is a stateless FastAPI service that sits between the system of record (D365 / ServiceNow) and a set of external reasoning and data services.



Actor	Interaction
Requester	Files the case in D365 / ServiceNow.
Support Engineer	Reads the AI note; clicks Accept/Reject and 👍/👎; sends customer emails.
Support Manager	Monitors the SLA-risk dashboard; approves suggested workflows.
D365 / ServiceNow	Source of webhook events, target of the AI note (system of record).

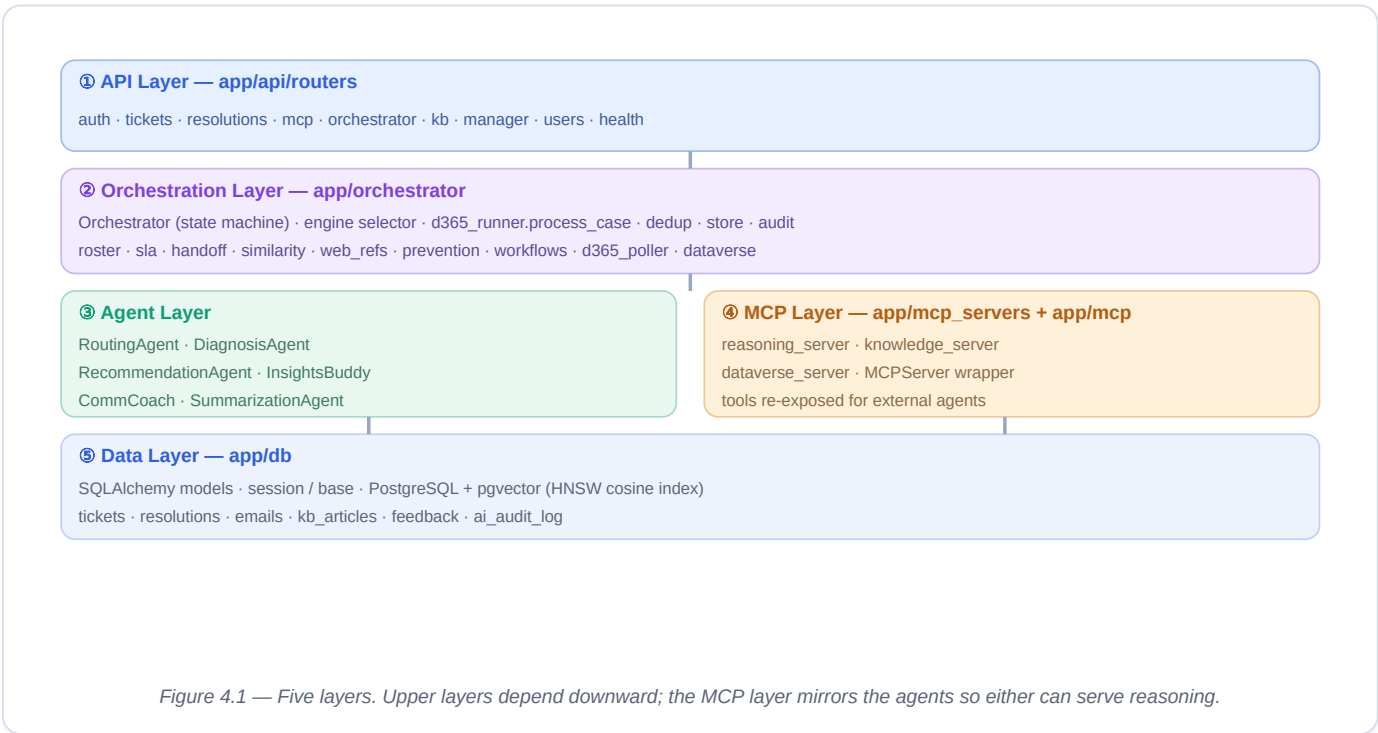
3. High-level architecture

The orchestrator classifies each event, selects a reasoning engine, runs the agent pipeline, evaluates the safety gate, composes one note, and audits every step.



Stateless by design. Two endpoints drive the ServiceNow flow — `/webhook` (event happened) and `/decision` (engineer responded); D365 adds `/d365-webhook` plus a background poller for fully hands-off automation.

4. Component & layer architecture



Layer	Responsibility	Key modules
① API	HTTP surface: auth, ticket CRUD, MCP analysis, webhooks, KB, manager dashboard	app/api/routers/*
② Orchestration	Event classify, dedupe, state machine, pipeline sequencing, engine selection, SLA/roster/handoff, note composition, safety net	app/orchestrator/*
③ Agents	Reasoning units: routing, diagnosis, recommendation, insights, email, summarization	app/agents/*
④ MCP	Every capability re-exposed as a discoverable MCP tool for external agents and the <code>mcp</code> engine	app/mcp_servers/*
⑤ Data	pgvector corpus of tickets, resolutions, KB, feedback, audit	app/db/*

5. Ticket lifecycle — state machine

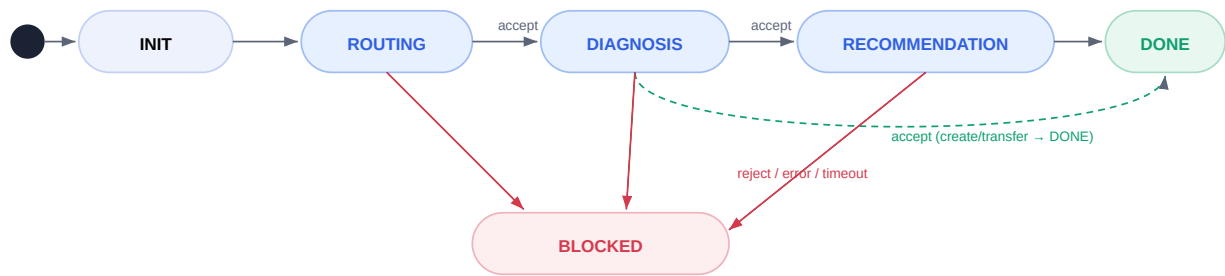


Figure 5.1 — Linear pipeline with a drop-to-BLOCKED on rejection, error, or timeout.

Event type	Pipeline	Use case
create	routing → diagnosis	New ticket
transfer	routing → diagnosis	Reassignment
reactivate	routing → diagnosis → recommendation	Reopened ticket

Each agent posts a JSON **advisory**; the engineer's **Accept** advances the pipeline, **Reject** drops to **BLOCKED** and flags the override for retraining. After an accepted recommendation, an optional **health-check probe** re-fires ~15 min later if symptoms persist (capped at 2 follow-ups).

6. End-to-end data flow (case → outcome)

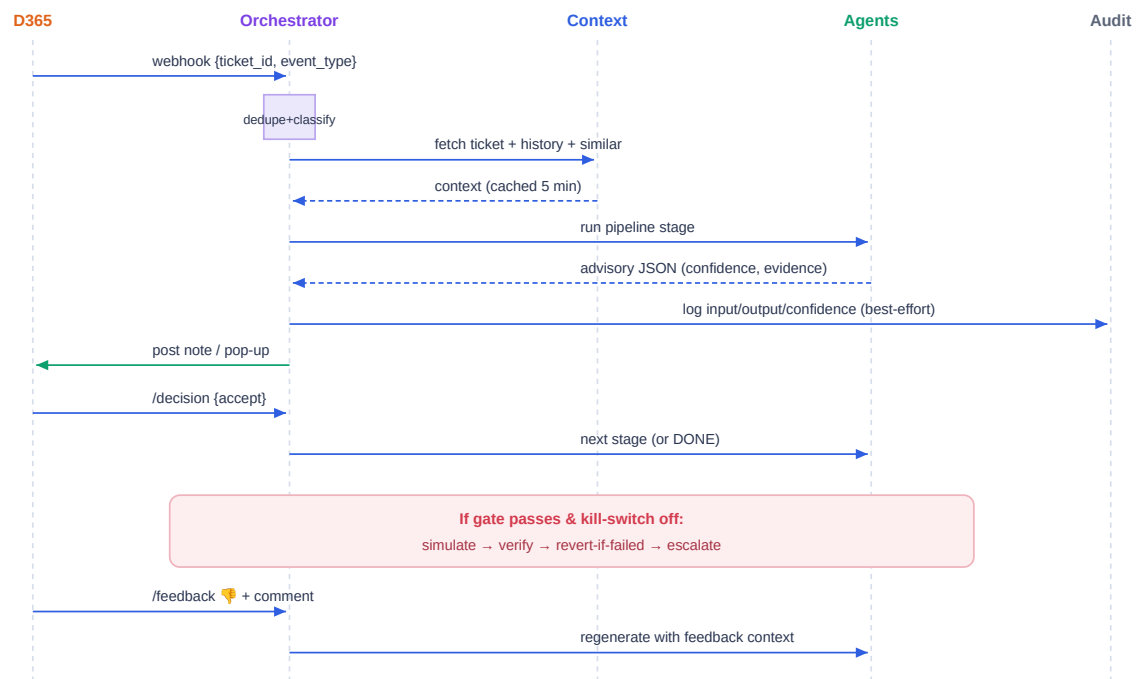


Figure 6.1 — Request/response sequence from webhook to feedback-driven regeneration.

1. **Dedupe first** — webhook + Power Automate retries + poller can all fire the same case within ~20s; Redis `SET NX` (10-min TTL) guarantees one note.
2. **Context is cached** — ticket + history cached 5 min per `ticket_id`; the event is always rebuilt fresh.
3. **Diagnosis is grounded** — cosine similarity over the `resolutions` corpus (pgvector), not free-form generation.
4. **References are allowlisted** — recommendation links validated against `trusted_domains.json` / Microsoft Learn only.
5. **Feedback loops back** — 🗨️ re-runs the relevant agent with the engineer's comment injected as extra context.

7. Dual reasoning engine (the fallback flag)

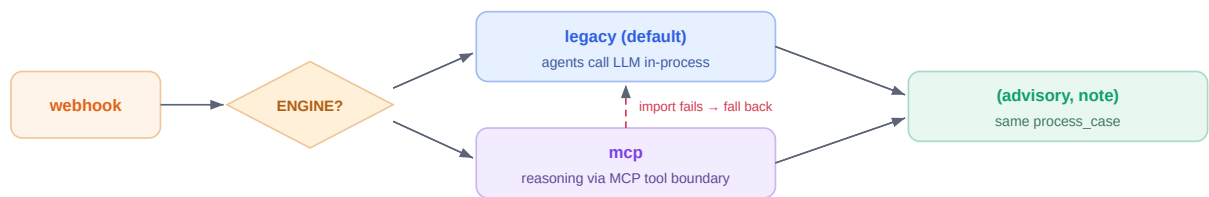


Figure 7.1 — `ENGINE=mcp` routes reasoning through MCP tools; failure falls back to legacy. Both reuse the same downstream code.

Both engines funnel through the **same** `d365_runner.process_case`, so orchestration, confidence blend, grounding cap, and note formatting **never diverge** — only *how each agent's answer is produced* differs. This lets MCP be adopted incrementally and rolled back instantly via one env flag, with zero behavioral drift.

8. Agents, roster & SLA

Agent	Responsibility	Output shape
RoutingAgent	Classify into a support team (10 K8s/Karpenter categories) + pick on-shift, skill-matched engineer	<code>recommended_team</code> , <code>assigned_engineer{}</code> , <code>confidence</code>
DiagnosisAgent	Root cause grounded in nearest historical cases	<code>root_cause</code> , <code>grounded</code> , <code>confidence</code>
RecommendationAgent	Hot Fix + Ultimate Fix + allowlisted reference links	<code>hot_fix{}</code> , <code>ultimate_fix{}</code> , <code>trusted_links[]</code>
InsightsBuddy	In-app analysis: similarity, KB match, recommended steps, root cause	<code>similar_tickets</code> , <code>recommended_steps</code> , <code>kb_recommendations</code>
CommCoach	Draft → edit → approve → send emails; log resolutions; flag stale comms	<code>Email(DRAFT APPROVED)</code> , <code>Resolution</code>
SummarizationAgent	1–3 sentence summary + error-code extraction	<code>summary</code> , <code>error_codes[]</code>

Shift-aware assignment. `roster.py` picks engineers in tiers (on-shift+specialty → on-call+specialty → any on-shift → any on-call → least-loaded w/ SLA-risk flag), respecting each engineer's own timezone. `sla.py` maps priority to Tier 1/2/3 (15 min-4 h / 60 min-8 h / 240 min-24 h), failing open to Tier 3.

9. MCP (Model Context Protocol) layer

Each agent capability is re-exposed as a discoverable MCP tool — serving both the in-process `mcp` engine and external LLM agents (Copilot Studio / Azure AI Foundry) over stdio or HTTP.

reasoning_server · 8104

`route_case` · `diagnose_case` · `recommend_case` — backed by the Routing/Diagnosis/Recommendation agents.

knowledge_server · 8102

`search_docs` — Microsoft Learn search filtered by the trusted-domain allowlist (no forums/blogs).

dataverse_server · 8101

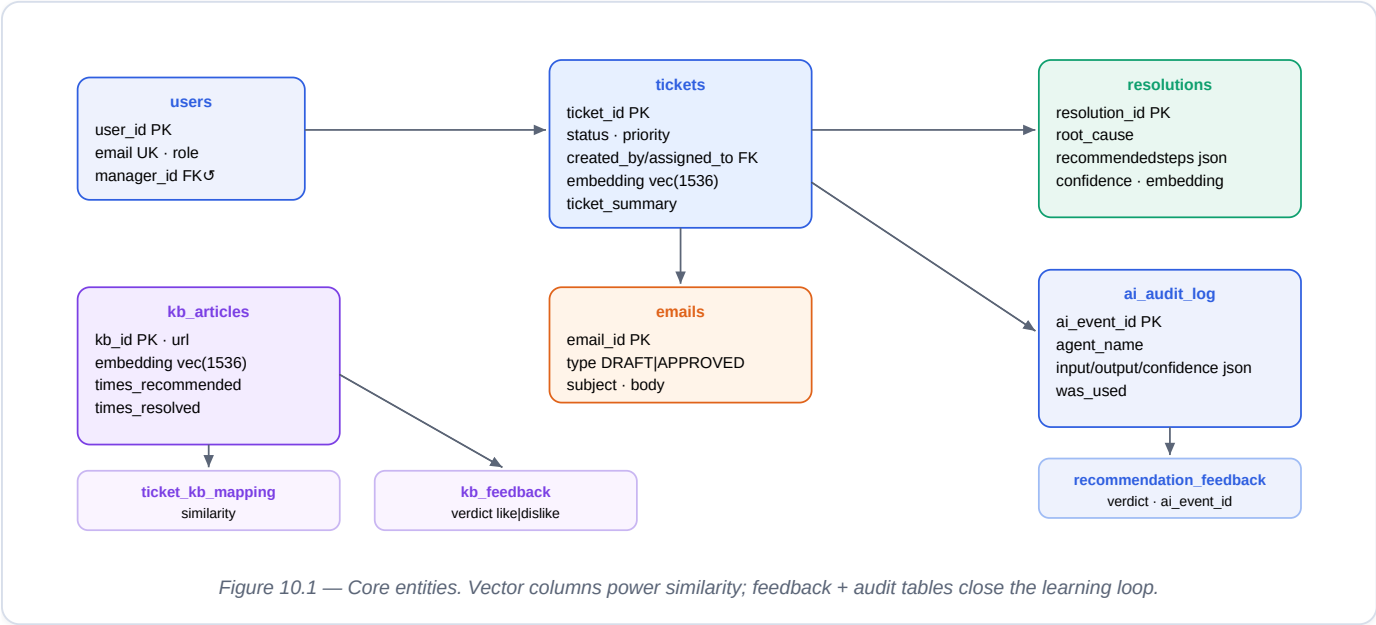
`list_cases` · `get_case` · `find_similar_cases` · `write_note` — backed by `DataverseClient`.

MCPServer wrapper

Composes InsightsBuddy + CommCoach for the in-app analysis and email workflow (`app/mcp/server.py`).

Servers run **in-process by default** (imported callables) or as **standalone processes** over stdio/HTTP — `_runtime.py` reads `MCP_TRANSPORT` / `MCP_HOST` / `MCP_PORT` — for out-of-process tool isolation.

10. Data model

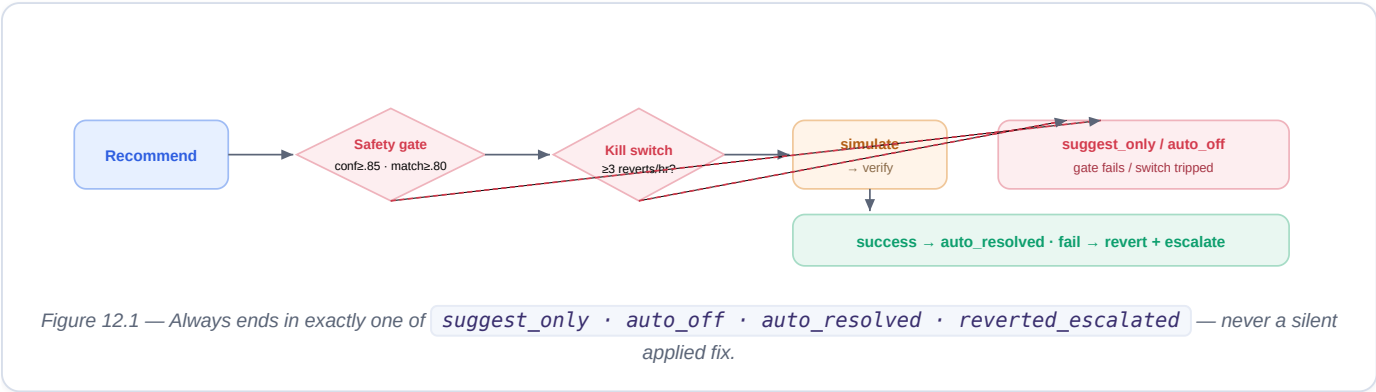


Outcome-based KB ranking. `times_recommended` / `times_resolved` / `total_resolution_hours` yield computed `success_rate` and `avg_resolution_hours`, so KB articles re-rank by *proven* value — not static metadata.

11. Integrations & graceful degradation

Dependency	Used for	If unavailable
Azure OpenAI (gpt-4o)	Agent reasoning (<code>llm.chat_json</code>)	Deterministic fallback responses
Azure OpenAI embeddings	Similarity search	Falls back to reference tickets
Dataverse Web API	Read cases / write notes (OAuth2)	<code>available()</code> → False; in-memory demo
Microsoft Learn	Reference links	Links omitted; recommendation still produced
PostgreSQL + pgvector	Persistence + vector search	Works without persistence/audit
Redis	Dedupe + state + context cache	In-memory dict fallback

12. Safety net — guarded auto-remediation



13. Deployment topology

Full app

`backend/app/main.py` — all routers + ML deps
(`requirements.txt`); the complete in-app experience.

Orchestrator-only

`orchestrator_server.py` — no torch/sentence-transformers;
slim always-on hosting (Render). Optional DB.

MCP: in-process

Default — tools imported as callables; lowest latency.

MCP: standalone

`mcp-service/main.py` ports 8101–8104 for tool isolation. Local dev via `docker-compose` (pgvector + redis).

14. Key design decisions & roadmap

Decision	Rationale
Dual engine, identical downstream logic	Adopt MCP incrementally, roll back instantly (env flag), zero behavioral drift.
Config externalized, thresholds in env	Runbooks/domains change often (ops content); safety thresholds stay visible in deploy config.
Verify-then-revert, not verify-then-trust	Auto-remediation is policed by its own kill switch — catching confident-but-wrong fixes.
Best-effort persistence	The case pipeline is the product; the audit trail must never be a single point of failure.
Grounded reasoning only	Diagnosis cites real past cases; references come from an allowlist — no hallucinated fixes/links.
Shift-aware assignment	The SLA clock cares whether the assignee is awake now, not the customer's timezone.
Stateless + optional stores	Runs with or without Postgres/Redis for low-friction environments.

Known gaps / roadmap

- `dataverse_server.py` remediation execution is **simulated** — real Dataverse/Graph/Defender API calls are Phase-3 placeholders.
- `app/services/` is empty — no custom services beyond agents/orchestrator yet.
- `tests/test_basic.py` is a placeholder; real coverage lives in `backend/app/orchestrator/test_orchestrator.py`.
- Bearer-token auth on orchestrator webhooks is planned before UAT (open in dev today).
- Next milestone: a real Power Automate "approve-and-execute" flow so managers approve suggested workflows — the orchestrator only ever *proposes*, never executes unattended.

In one line: Quest is a resilient, grounded, audit-first triage pipeline where AI *suggests* by default and only *acts* behind a confidence gate, a precedent gate, a kill switch, and a verify-then-revert net — with a dual engine and MCP tool layer that make the reasoning both swappable and externally reusable.